

## Linear algebra.

- Q1. Why logical cache faster than physical cache?
- logical cache is the virtual address of the memory which is closer to MMU (memory management unit) and it does not have to go from MMU due to what it does not have to work extra and the reason it is faster than physical cache, whereas physical cache first have to go from MMU and the reason is slightly less faster than logical cache.

RAM (Random access memory) →

RAM (V → Volatile)

ROM (N → Non-volatile)

for cache → for RAM or main memory.

SRAM vs DRAM

- Static RAM and Dynamic RAM are two types of semiconductor memory that differ in size, shape, speed, cost and use.
- Static RAM: SRAM stores each bit of data using a bistable flip-flop circuit, which can hold data as long as the power is supplied. It does not require periodic refreshing, making it faster and more reliable in terms of access time. Therefore, SRAM is commonly used in cache memory within a CPU where high speed is essential.
- Dynamic: Dynamic RAM stores data in capacitors which gradually lose charge over time. Hence it requires periodic refreshing to maintain stored data, leading to slower performance than SRAM. It is denser, cheaper and has higher storage capacity.

- Q3. When a new block of must be loaded in the cache given it is replacing an existing block, in what case it would be required to write something to the main memory?
- When a new block of is loaded into the cache, replacing an existing block, it is necessary to write the old block to main memory if that block has been modified while in the cache.

This situation occurred under the write-back policy. In this policy, any change made to the cache block are not immediately written to main memory. Instead a dirty bit is set to indicate that the cache block has been modified.

So when that block is later replaced (evicted), the system writes the updated data back to main memory to maintain consistency.

- Q4. Let say you want to transfer 7 bit data with even Parity, and your data bits are 0110010, what will the final data look like.

0110010 → 7 Bit → odd Parity -

add 1 to front.

1 0 1 1 0 0 1 0

check location of 1. 2, 3, 6<sup>th</sup> Place  
number of 1s → ③ → (odd)